

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

_____ * _____



SOICT

BÁO CÁO

Đề tài:

**HỆ THỐNG AIoT HỖ TRỢ HỌC TẬP
SỬ DỤNG THIẾT BỊ ĐO SÓNG NÃO VÀ CAMERA**

Giảng viên hướng dẫn: Trịnh Văn Chiến
Nguyễn Tuấn Đạt

Mã học phần: IT3943

Tên học phần: Project 3

Học kỳ: 2025.2

Sinh viên thực hiện: Lê Hồng Sơn - 20225389

Hà Nội, tháng 7 năm 2026

MỤC LỤC

LỜI NÓI ĐẦU.....	4
I. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	4
1. Tổng quan về hệ thống AIoT hỗ trợ học tập.....	4
2. Điện não đồ EEG và vi mạch TGAM.....	4
3. Thị giác máy tính với MediaPipe và DeepFace.....	5
4. Công nghệ phần mềm.....	6
II. PHÂN TÍCH YÊU CẦU HỆ THỐNG.....	6
1. Yêu cầu chức năng.....	6
2. Yêu cầu phi chức năng.....	7
3. Mô hình hóa chức năng bằng biểu đồ usecase.....	8
4. Đặc tả chi tiết Use Case cốt lõi.....	8
III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	13
1. Kiến trúc tổng thể.....	13
2. Luồng dữ liệu thời gian thực.....	14
3. Thiết kế dữ liệu.....	14
4. Thiết kế thuật toán hợp nhất cảm biến.....	15
5. Thiết kế các chỉ số hiển thị.....	16
IV. TRIỂN KHAI VÀ THỰC NGHIỆM.....	17
1. Triển khai tầng Edge.....	17
2. Triển khai Backend.....	17
3. Triển khai Frontend.....	18
4. Huấn luyện mô hình.....	18
5. Kết quả đánh giá.....	19
V. KẾT LUẬN.....	20

1.	Kết quả đạt được	20
2.	Hạn chế.....	20
3.	Hướng phát triển.....	21
	TÀI LIỆU THAM KHẢO	22

LỜI NÓI ĐẦU

Sự phát triển của học trực tuyến, làm việc từ xa và các nền tảng số giúp người học tiếp cận tài nguyên linh hoạt hơn, nhưng đồng thời làm tăng khó khăn trong việc tự duy trì sự tập trung. Các phương pháp đánh giá bằng bảng hỏi hoặc quan sát thủ công có độ trễ cao và chịu ảnh hưởng đáng kể bởi nhận định chủ quan. Vì vậy, đề tài xây dựng một hệ thống AIoT có khả năng thu thập đồng thời tín hiệu điện não và hành vi thị giác nhằm hỗ trợ theo dõi trạng thái học tập theo thời gian thực.

Hệ thống sử dụng thiết bị TGAM để thu nhận chỉ số eSense Attention, Meditation, chất lượng tín hiệu và năng lượng tám dải sóng não; đồng thời sử dụng webcam máy tính, MediaPipe Face Mesh và DeepFace để xác định hướng đầu, hướng mắt và biểu cảm khuôn mặt. Dữ liệu được xử lý tại tầng biên Python, gửi tới máy chủ Node.js qua Socket.IO, lưu trữ trên MongoDB và hiển thị trên React Dashboard.

Mục tiêu của đề tài gồm: xây dựng luồng dữ liệu khép kín từ cảm biến đến giao diện; thử nghiệm mô hình Random Forest phân loại trạng thái “Tập trung” và “Sao nhãng”; hỗ trợ thu thập, gán nhãn và xuất dữ liệu phục vụ huấn luyện; thiết kế giao diện trực quan gồm sóng EEG thô, năng lượng các dải sóng, camera và trạng thái nhận thức. Hệ thống chỉ có vai trò hỗ trợ kỹ thuật và không được sử dụng thay thế cho chẩn đoán y khoa hoặc đánh giá tâm lý chuyên môn.

I. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

1. Tổng quan về hệ thống AIoT hỗ trợ học tập

AIoT là sự kết hợp giữa Internet of Things và các thuật toán trí tuệ nhân tạo. Trong đề tài, lớp IoT chịu trách nhiệm kết nối thiết bị EEG, camera, máy chủ và trình duyệt; lớp AI xử lý đặc trưng thị giác và phân loại trạng thái nhận thức. Kiến trúc được tổ chức theo ba tầng: Edge, Backend và Frontend, giúp tách biệt tác vụ phân cứng, điều phối dữ liệu và hiển thị.

2. Điện não đồ EEG và vi mạch TGAM

Điện não đồ (EEG) ghi nhận dao động điện sinh học tạo bởi hoạt động của các tế bào thần kinh. Thiết bị sử dụng trong đề tài là cảm biến một kênh vùng trán tích hợp vi mạch TGAM. TGAM gửi dữ liệu qua cổng Serial Bluetooth COM14 với tốc độ 57.600 baud theo giao thức ThinkGear.

2.1. Các dải sóng não

Các dải sóng EEG sử dụng trong hệ thống

Dải sóng	Khoảng tần số tham khảo	Ý nghĩa thường gặp trong nghiên cứu
Delta	0,5 - 4 Hz	Hoạt động chậm; thường nổi bật trong giấc ngủ sâu.
Theta	4 - 8 Hz	Thư giãn sâu, buồn ngủ hoặc tải nhận thức trong một số nhiệm vụ.
Alpha	8 - 13 Hz	Trạng thái thư giãn tỉnh táo, giảm kích thích thị giác.
Beta	13 - 30 Hz	Hoạt động nhận thức, tập trung và xử lý thông tin.
Gamma	30 - 50 Hz	Hoạt động xử lý tích hợp ở tần số cao; dễ bị ảnh hưởng bởi nhiễu cơ.

Các diễn giải trên chỉ mang tính tham khảo. Một dải sóng đơn lẻ không đủ để kết luận trạng thái nhận thức; kết quả còn phụ thuộc vị trí điện cực, chất lượng tiếp xúc, chuyển động và đặc điểm từng người.

2.2. Giao thức ThinkGear và gói ASIC_EEG_POWER

Mỗi gói ThinkGear có hai byte đồng bộ 0xAA 0xAA, một byte độ dài, phần payload và một byte checksum. Bộ đọc TGAM loại bỏ các gói có checksum không hợp lệ trước khi giải mã dữ liệu.

Các mã ThinkGear được xử lý

Mã	Dữ liệu	Vai trò trong đề tài
0x02	Poor Signal	Đánh giá chất lượng tiếp xúc điện cực; 0 là tốt, 200 là off-head hoặc không sử dụng được.
0x04	Attention	Chỉ số eSense chú ý, khoảng 0-100.
0x05	Meditation	Chỉ số eSense thư giãn, khoảng 0-100.
0x80	Raw EEG	Mẫu EEG thô 16 bit có dấu, dùng để vẽ waveform.
0x83	ASIC_EEG_POWER	Tám giá trị năng lượng: Delta, Theta, Low/High Alpha, Low/High Beta, Low/Mid Gamma.

Vì mạch TGAM đã cung cấp sẵn năng lượng của tám dải trong gói 0x83. Vì vậy, biểu đồ năng lượng dải sóng của hệ thống sử dụng trực tiếp các giá trị này mà không cần tự tính FFT.

3. Thị giác máy tính với MediaPipe và DeepFace

3.1. MediaPipe Face Mesh

MediaPipe Face Mesh cung cấp các điểm mốc khuôn mặt theo thời gian thực. Hệ thống sử dụng các điểm ở mũi, má, khóe mắt và móng mắt để ước lượng ba trạng thái hướng đầu (nhìn thẳng, quay trái, quay phải) và ba trạng thái ánh mắt (nhìn thẳng, liếc trái, liếc phải). Khi không phát hiện được khuôn mặt hoặc mắt, hệ thống trả về trạng thái tương ứng để tránh suy luận từ dữ liệu không hợp lệ.

3.2. DeepFace

DeepFace được chạy trong một luồng nền riêng để phân tích biểu cảm mà không chặn luồng camera chính. Các nhãn được ánh xạ sang tiếng Việt gồm: Vui vẻ, Buồn bã, Tức giận, Ngạc nhiên, Sợ hãi, Khó chịu và Bình thường.

4. Công nghệ phần mềm

4.1. Tầng Edge Python

Python được sử dụng để giao tiếp Serial, xử lý camera, nạp mô hình học máy và gửi dữ liệu Socket.IO. Các thư viện chính gồm pyserial, OpenCV, MediaPipe, DeepFace, pandas, scikit-learn, joblib và python-socketio.

4.2. Backend Node.js và MongoDB

Backend sử dụng Node.js, Express và Socket.IO để nhận event sensor_data từ Edge, phát lại dữ liệu tới trình duyệt, cung cấp REST API và lưu telemetry hoặc label event vào MongoDB Atlas. Cơ chế bất đồng bộ phù hợp với luồng dữ liệu thời gian thực có nhiều gói nhỏ.

4.3. Frontend React và Recharts

Frontend React hiển thị các chỉ số EEG, camera, trạng thái thị giác, dự đoán của mô hình và các biểu đồ. Recharts được sử dụng cho biểu đồ đường và biểu đồ cột; Socket.IO Client nhận dữ liệu mà không cần tải lại trang.

4.4. Mô hình Random Forest

Random Forest là mô hình tổ hợp nhiều cây quyết định. Mỗi cây học trên một phần dữ liệu và tập đặc trưng khác nhau; kết quả cuối cùng được xác định theo biểu quyết. Đề tài sử dụng 100 cây quyết định để phân loại hai lớp “Tập trung” và “Sao nhãng”.

II. PHÂN TÍCH YÊU CẦU HỆ THỐNG

1. Yêu cầu chức năng (Functional Requirements)

- **RF-01 (Thu thập dữ liệu biên):** Hệ thống phải đọc được liên tục các byte dữ liệu điện não từ cổng Serial COM và bắt được luồng hình ảnh từ Webcam.
- **RF-02 (Dự đoán trạng thái):** Hệ thống phải ứng dụng mô hình AI nạp sẵn để chẩn đoán trạng thái "Tập trung" hoặc "Sao nhãng".

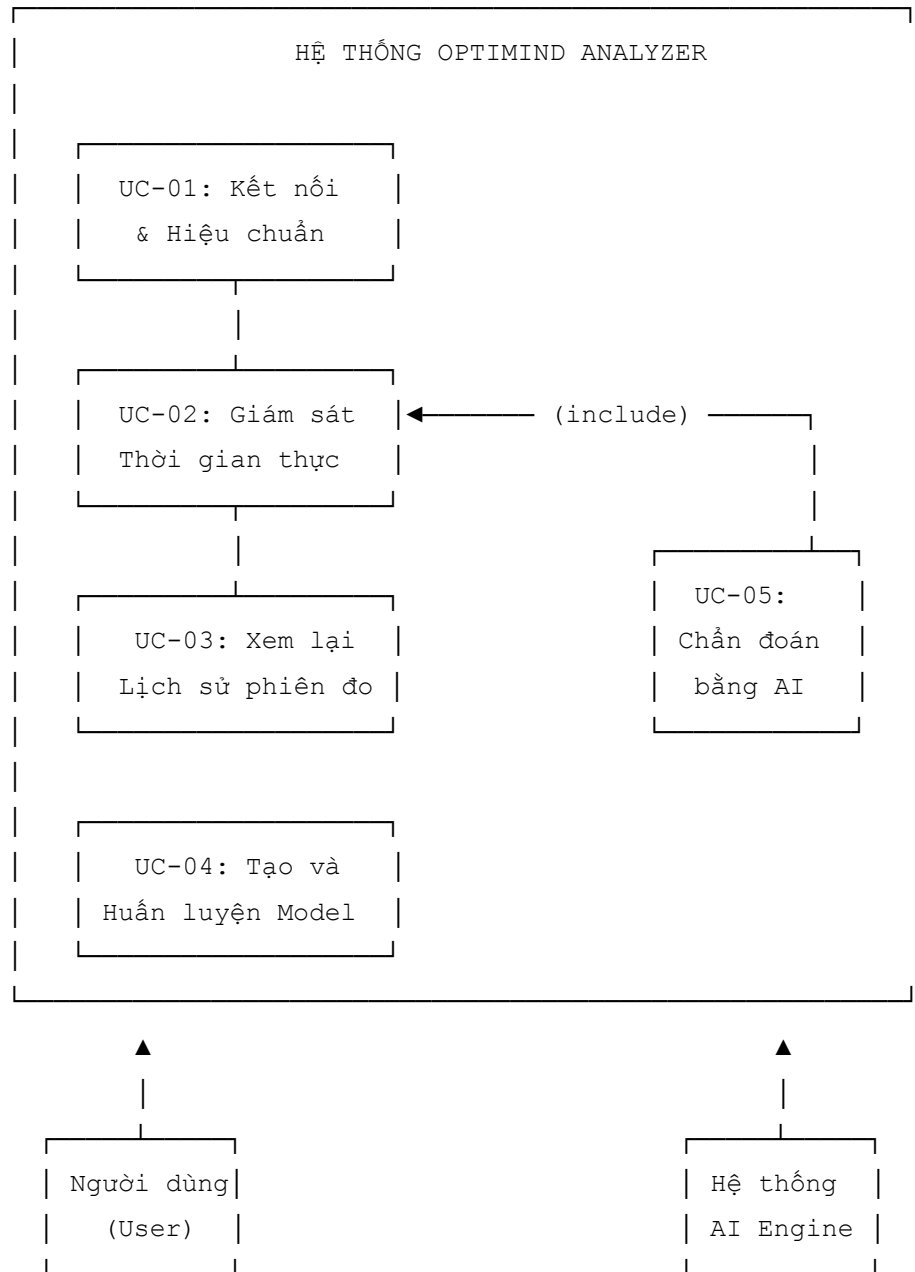
- **RF-03 (Truyền tải real-time):** Hệ thống phải đóng gói payload và đẩy lên Web Dashboard thông qua giao thức Socket với chu kỳ 0.04 giây (25 FPS).
- **RF-04 (Lưu trữ lịch sử):** Hệ thống phải lưu được thông tin phiên đo (telemetries) và nhãn sự kiện (labels) vào cơ sở dữ liệu để phục vụ hậu kiểm và tái huấn luyện mô hình.

2. Yêu cầu phi chức năng (Non-functional Requirements)

- **NRF-01 (Độ trễ):** Độ trễ truyền tải dữ liệu từ cảm biến lên giao diện hiển thị không vượt quá 100ms.
- **NRF-02 (Hiệu năng):** Luồng xử lý camera không được gây hiện tượng sụt giảm khung hình (Drop FPS), duy trì tối thiểu 25 FPS trên các cấu hình máy tính cá nhân tiêu chuẩn.
- **NRF-03 (Tính toàn vẹn):** Hệ thống phải có cơ chế kiểm tra lỗi checksum đối với gói tin sóng nã để loại bỏ dữ liệu rác trước khi đưa vào mô hình AI.

3. Mô hình hóa chức năng bằng biểu đồ Use Case

Dưới đây là cấu trúc biểu đồ Use Case mô tả mối quan hệ giữa các Tác nhân (Actors) và các Chức năng (Use Cases) của hệ thống:



Biểu đồ usecase tổng quan

4. Đặc tả các Use Case cốt lõi

Đặc tả Use Case UC-01: Kết nối và Hiệu chuẩn thiết bị (Handshake)

- **Actor chính:** Người dùng (User).

- **Actor hỗ trợ:** Khởi xử lý biên (TGAM Reader).
- **Mục đích:** Khởi tạo kết nối vật lý nối tiếp, bắt tay giao tiếp đồng bộ giữa thiết bị phần cứng MindLink và ứng dụng biên Python, kiểm tra chất lượng thu phát sóng sinh học.
- **Điều kiện tiên quyết:** Thiết bị đeo đầu MindLink đã được bật nguồn và đã ghép đôi Bluetooth thành công với máy tính; cổng COM ảo được HĐH định danh.
- **Điều kiện sau khi hoàn thành:** Luồng kết nối Serial được giữ mở ổn định, sẵn sàng cung cấp các byte dữ liệu thô liên tục cho luồng chính.

Luồng sự kiện chính:

1. Người dùng khởi chạy chương trình tại Edge Node (python main_fusion.py).
2. Khởi TGAMReader được khởi tạo với cấu hình mặc định: cổng COM14, tốc độ baud_rate=57600.
3. Hệ thống kích hoạt một luồng xử lý độc lập ngầm để thực hiện tác vụ mở cổng Serial.
4. Hệ thống thực hiện quét dòng dữ liệu truyền về từ thiết bị.
5. Khi phát hiện chuỗi byte tiền định dạng liên tiếp 0xAA 0xAA, hệ thống đọc byte tiếp theo để xác định độ dài gói tin (plen).
6. Hệ thống thực hiện đọc toàn bộ gói Payload và tính toán mã kiểm tra toàn vẹn Checksum.
7. Hệ thống đối chiếu kết quả tính được với byte Checksum cuối cùng do phần cứng gửi về để xác thực.
8. Hệ thống trích xuất byte chất lượng tín hiệu, ghi nhận giá trị signal == 0 (Trạng thái tiếp xúc điện cực lý tưởng).
9. Màn hình console xuất thông báo xác nhận: *"Đã kết nối thành công! Sẵn sàng nhận sóng não"*.

Luồng sự kiện thay thế / Ngoại lệ:

- **Ngoại lệ [Cổng COM bị chiếm dụng]:** Nếu hệ thống không thể mở cổng COM ảo, hệ thống tự động rơi vào trạng thái chờ kết nối lại sau mỗi chu kỳ 5 giây.

- **Ngoại lệ [Lỗi toàn vẹn Checksum]:** Nếu dữ liệu truyền qua sóng Bluetooth bị nhiễu môi trường dẫn đến tính toán Checksum sai lệch, hệ thống lập tức hủy bỏ toàn bộ gói Payload đó và tiếp tục quét nhằm tránh nạp dữ liệu rác.

Đặc tả Use Case UC-02: Giám sát Thời gian thực (Real-time Monitoring)

- **Actor chính:** Người dùng (User).
- **Actor hỗ trợ:** Hệ thống AI Engine.
- **Mục đích:** Cho phép người dùng theo dõi trực quan trạng thái tâm lý nhận thức và các dải sóng não thô dạng biểu đồ động khi đang làm việc.
- **Điều kiện tiên quyết:** Thiết bị đeo đầu TGAM đã bật và kết nối thành công qua cổng COM; ứng dụng backend server đang hoạt động.
- **Điều kiện sau khi hoàn thành:** Toàn bộ dữ liệu của phiên được lưu trữ thành công vào cơ sở dữ liệu MongoDB.

Luồng sự kiện chính:

1. Người dùng truy cập vào trang MasterDashboard trên trình duyệt Web.
2. Frontend thiết lập kết nối Socket.io tới Server Node.js.
3. Tại Edge Node, file main_fusion.py khởi chạy luồng đọc camera và sóng não.
4. Tầng Edge liên tục gửi gói tin ui_payload lên Server qua Socket phát tán.
5. Server trung chuyển gói tin xuống Frontend Client.
6. Frontend nhận dữ liệu, thực hiện thuật toán hợp nhất (Sensor Fusion), cập nhật trạng thái nhận thức lên 4 thanh sắc màu AI (Focus, Relaxation, Stress, Fatigue) và vẽ lại biểu đồ.
7. Hệ thống lặp lại bước 4-6 liên tục mỗi 0.04 giây.

Luồng sự kiện thay thế / Ngoại lệ:

- **Ngoại lệ [Tín hiệu nhiễu vật lý]:** Nếu chất lượng tiếp xúc da đầu kém, chip TGAM trả về mã lỗi signal > 50. Hệ thống tạm dừng đẩy các chỉ số sóng não vào bộ phân lớp AI để tránh sai số chẩn đoán.
- **Ngoại lệ [Mất kết nối phần cứng]:** Nếu thiết bị ngắt kết nối, biến signal nhận giá trị cố định 200. Hệ thống chuyển giao diện hiển thị sang trạng thái "System Offline" và hiển thị cảnh báo "NO DATA".

Đặc tả Use Case UC-03: Xem lại Lịch sử phiên đo (Review History)

- **Actor chính:** Người dùng (User).
- **Mục đích:** Cho phép người dùng truy vấn, tra cứu lịch sử và hiển thị trực quan biểu đồ tiến trình của các phiên đo nhận thức trong quá khứ đã được lưu vào cơ sở dữ liệu.
- **Điều kiện tiên quyết:** Máy chủ REST API Node.js và hệ quản trị MongoDB Atlas đang hoạt động bình thường trên mạng.
- **Điều kiện sau khi hoàn thành:** Dữ liệu lịch sử được kết xuất thành công, hiển thị tường minh dưới dạng bảng dữ liệu hoặc đồ thị xu hướng nhận thức.

Luồng sự kiện chính:

1. Người dùng bấm chọn tab "Lịch sử phiên đo" trên giao diện Web Dashboard.
2. Frontend phát một yêu cầu HTTP GET Request tới Endpoint REST API của máy chủ Backend.
3. Backend tiếp nhận yêu cầu, thực hiện truy vấn cơ sở dữ liệu MongoDB.
4. Dữ liệu được sắp xếp giảm dần theo thời gian và giới hạn số lượng dòng để tối ưu hiệu năng.
5. MongoDB trả về mảng danh sách các Document chứa thông số sóng não và hành vi dưới định dạng JSON.
6. Backend đóng gói dữ liệu và gửi phản hồi HTTP Status Code 200 về cho Frontend.
7. Frontend Client tiếp nhận mảng JSON, nạp dữ liệu vào các cấu phần đồ họa chuyên biệt.
8. Màn hình hiển thị bảng thống kê chi tiết kèm đồ thị đường mô tả xu hướng biến thiên chỉ số tập trung.

Đặc tả Use Case UC-04: Tạo và Huấn luyện Model AI (Create and Train Model)

- **Actor chính:** Người dùng (User).
- **Actor hỗ trợ:** Hệ thống AI Engine.

- **Mục đích:** Kết xuất tập dữ liệu thô từ database, thực hiện tiền xử lý mã hóa toán học và chạy thuật toán Random Forest để sinh ra file mô hình AI phục vụ chẩn đoán.
- **Điều kiện tiên quyết:** Số lượng bản ghi dán nhãn tích lũy trong MongoDB đạt số lượng tối thiểu (> 100 mẫu). Môi trường Python biên đã cài đặt đủ thư viện.
- **Điều kiện sau khi hoàn thành:** File mô hình optimind_ai_model.pkl và các bộ mã hóa được tạo mới hoặc cập nhật ghi đề thành công vào hệ thống.

Luồng sự kiện chính:

1. Người dùng kích hoạt chương trình huấn luyện bằng lệnh biên dịch (python train_model.py).
2. Hệ thống thực hiện kết nối cơ sở dữ liệu, đọc dữ liệu và xuất thành file cấu trúc phẳng CSV.
3. Tập lệnh Python nạp file CSV thành cấu trúc dữ liệu DataFrame nhờ thư viện Pandas.
4. Hệ thống chuyển đổi tự động các cột dữ liệu chuỗi (biểu cảm khuôn mặt, hướng ánh mắt, hướng đầu) sang dạng mã số nguyên liên tục.
5. Hệ thống chia tách tập dữ liệu: Các biến độc lập (Features) gồm 9 trường dữ liệu cảm biến; Biến phụ thuộc (Target) là nhãn "Tập trung" hoặc "Sao nhãng".
6. Hệ thống thực hiện phân rã tập dữ liệu: 80% cho quá trình Học (Train Set) và 20% cho quá trình Kiểm thử (Test Set).
7. Hệ thống khởi tạo đối tượng RandomForestClassifier và tiến hành huấn luyện.
8. Sau khi thuật toán hội tụ, hệ thống nạp tập dữ liệu Test để đánh giá chẩn đoán độc lập.
9. Hệ thống tính toán và in ra màn hình báo cáo hiệu năng.
10. Hệ thống đóng gói mô hình thành file nhị phân tĩnh lưu trữ xuống đĩa cứng.

Đặc tả Use Case UC-05: Chẩn đoán bằng AI (AI Diagnosis)

- **Actor chính:** Hệ thống AI Engine.
- **Mục đích:** Tiếp nhận vectơ dữ liệu thô tích hợp từ sóng não và camera ở mỗi chu kỳ quét thời gian thực, thực hiện suy luận trích xuất nhãn phân loại tự động.

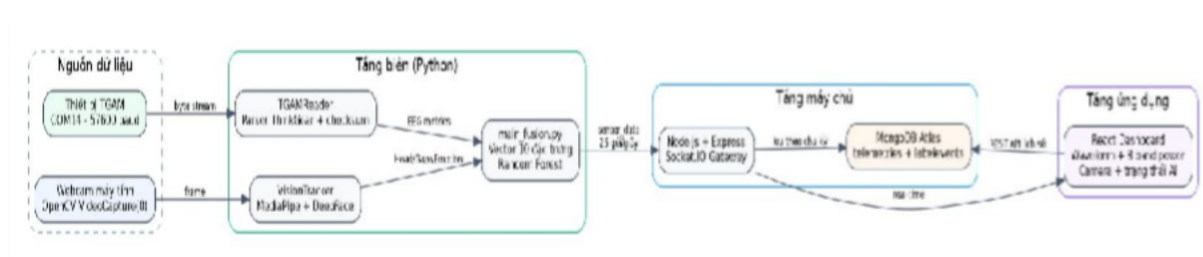
- **Điều kiện tiên quyết:** Khối Edge Node đã nạp thành công các file mô hình nhị phân .pkl vào bộ nhớ RAM. Dữ liệu thô truyền về hợp lệ.
- **Điều kiện sau khi hoàn thành:** Biến kết quả được gán nhãn chính xác chuỗi kết quả chẩn đoán trước khi đóng gói payload gửi lên mạng.

Luồng sự kiện chính:

1. Use Case này được gọi tự động bên trong vòng lặp của cấu phần main_fusion.py theo chu kỳ mỗi 0.04 giây.
2. Hệ thống thu thập các giá trị sóng não thô hiện tại và chuỗi trạng thái ngoại vi hiện tại.
3. Hệ thống nạp các chuỗi đặc trưng chữ vào các bộ mã hóa đã tải sẵn để nhận về các mã số tương ứng.
4. Hệ thống tổ chức dữ liệu thành một mảng Vector Đặc trưng gồm 10 phần tử theo trật tự nghiêm ngặt.
5. Hệ thống gọi hàm dự đoán của mô hình AI: model.predict().
6. Mô hình toán học thực hiện đánh giá và trả về kết quả lớp nhãn dự đoán.
7. Giá trị chuỗi kết quả được gán trực tiếp vào thuộc tính final_state bên trong cấu trúc gói tin.

III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

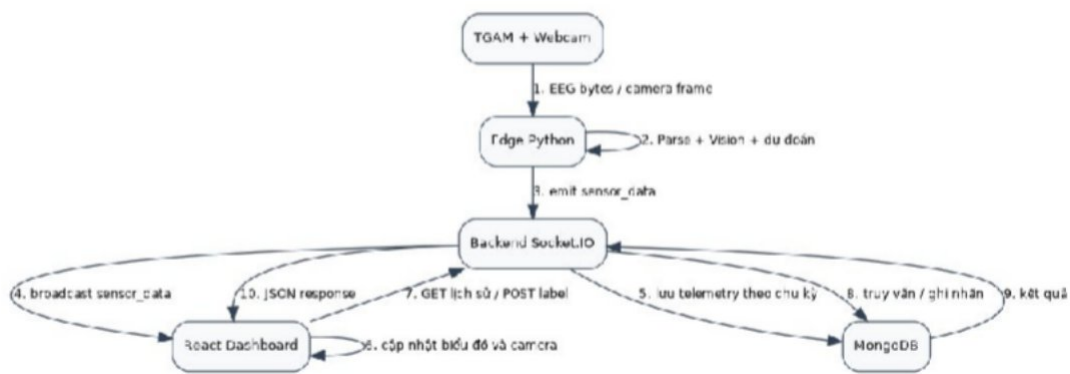
1. Kiến trúc tổng thể



Kiến trúc tổng thể hệ thống AIoT

Tầng Edge thực hiện các tác vụ cần độ trễ thấp: đọc cảm biến, xử lý thị giác và suy luận mô hình. Backend đóng vai trò gateway, lưu trữ và cung cấp API. Frontend đảm nhận trực quan hóa và tương tác với người dùng. Cách phân tầng này giúp bảo trì từng phần độc lập và hạn chế việc đưa xử lý nặng vào trình duyệt.

2. Luồng dữ liệu thời gian thực



Luồng trao đổi dữ liệu giữa thiết bị, Edge, Backend, Frontend và MongoDB

Event `sensor_data` được phát theo nhịp mục tiêu 0,04 giây. Dữ liệu EEG và trạng thái thị giác xuất hiện trong mỗi gói; ảnh Base64 chỉ được gán vào khoảng 10 gói mỗi giây. Đây là cơ chế lấy mẫu xuống ảnh camera trên cùng một kênh Socket.IO, giúp waveform cập nhật mượt nhưng giảm băng thông hình ảnh. Frontend phải bỏ qua frame rỗng và giữ lại ảnh hợp lệ gần nhất.

```
if (data.frame)
{
  setCameraFrame(data.frame);
}
```

3. Thiết kế dữ liệu

Dữ liệu thời gian thực được đóng gói trong payload JSON gồm bốn nhóm: `eeg`, `raw_values`, `vision` và `final_state`. Backend có thể lưu bản tóm tắt theo chu kỳ thay vì lưu toàn bộ 25 gói mỗi giây để giảm dung lượng cơ sở dữ liệu.

```
{
  "eeg": {
    "signal": 0,
    "attention": 78,
    "meditation": 61,
    "delta": 120000,
    "theta": 84000,
    "low_alpha": 41000,
```

```

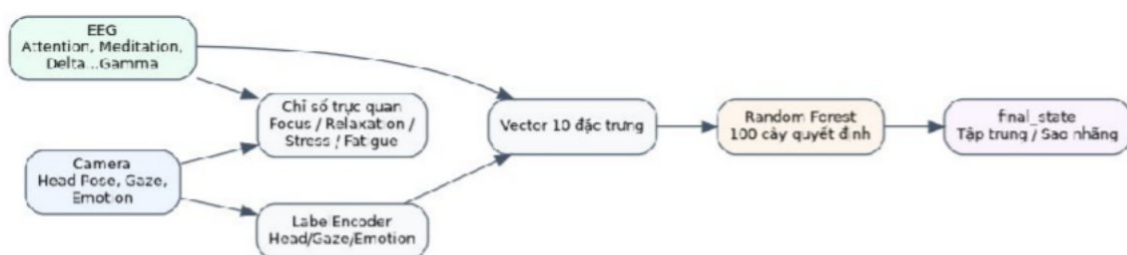
"high_alpha": 47000,
"low_beta": 90000,
"high_beta": 98000,
"low_gamma": 16000,
"mid_gamma": 14000
},
"raw_values": [ ... ], "vision":
{
  "head_pose_state": "Nhìn thẳng",
  "gaze_state": "Nhìn thẳng",
  "emotion": "Bình thường",
  "person_id": "USER-01"
},
"final_state": "Tập trung",
"frame": "<base64 hoặc chuỗi rỗng>"
}

```

Thiết kế dữ liệu MongoDB

Collection	Trường chính	Mục đích
telemetries	timestamp, person_id, eeg, vision, final_state	Lưu dữ liệu đo đã lấy mẫu để xem lịch sử và xuất báo cáo.
labelevents	session_id, taskName, reactionTime, isCorrect, label	Lưu nhãn từ phiên thử động hoặc bài kiểm tra nhận thức.

4. Thiết kế thuật toán hợp nhất cảm biến



Lưu ý xử lý đặc trưng và dự đoán

Mô hình tại Edge sử dụng vector 10 đặc trưng theo đúng thứ tự huấn luyện. Ba đặc trưng chữ được chuyển sang mã số bằng LabelEncoder. Nếu một nhãn mới không tồn tại trong encoder, cách gán mặc định về 0.

Vector đặc trưng đầu vào mô hình

STT	Đặc trưng	Nguồn
1	Attention	TGAM 0x04
2	Meditation	TGAM 0x05
3	Alpha = Low Alpha + High Alpha	TGAM 0x83
4	Beta = Low Beta + High Beta	TGAM 0x83
5	Theta	TGAM 0x83
6	Delta	TGAM 0x83
7	Gamma = Low Gamma + Mid Gamma	TGAM 0x83
8	Head Pose	MediaPipe
9	Gaze	MediaPipe
10	Emotion	DeepFace

5. Thiết kế các chỉ số hiển thị

5.1. Chỉ số *Focus*, *Relaxation*, *Stress* và *Fatigue*

Ngoài final_state của Random Forest, Frontend tính bốn chỉ số heuristic nhằm hỗ trợ trực quan hóa. Các công thức này không phải chỉ số y khoa đã được kiểm định và không được dùng để chẩn đoán.

- $\text{Focus} = 0,6 \times \text{Attention} + 0,2 \times \text{điểm hướng đầu} + 0,2 \times \text{điểm hướng mắt}$.
- $\text{Relaxation} = 0,8 \times \text{Meditation} + 0,2 \times \text{điểm cảm xúc thư giãn}$.
- $\text{Stress} = \max(0, \text{Attention} - \text{Meditation})$, có thể cộng hệ số khi biểu cảm thuộc nhóm tiêu cực.

- Fatigue = 100 - Attention, chỉ là đại lượng tham khảo. Nhận diện mệt mỏi đáng tin cậy cần thêm EAR, PERCLOS, MAR và head pitch.

5.2. Tỷ lệ năng lượng các dải sóng

TGAM trả về tám giá trị ASIC_EEG_POWER. Để hiển thị năm nhóm chính, hệ thống tính Alpha tổng, Beta tổng và Gamma tổng, sau đó chuẩn hóa theo tổng năng lượng tức thời.

```
alpha = low_alpha + high_alpha
beta = low_beta + high_beta
gamma = low_gamma + mid_gamma
P_total = delta + theta + alpha + beta + gamma ratio_band
= band / P_total * 100
```

IV. TRIỂN KHAI VÀ THỰC NGHIỆM

1. Triển khai tầng Edge

1.1. TGAMReader

TGAMReader mở COM14 ở 57.600 baud trong một daemon thread. Vòng đọc tìm hai byte đồng bộ 0xAA, đọc độ dài payload, kiểm tra checksum và giải mã các code 0x02, 0x04, 0x05, 0x80 và 0x83. raw_buffer hiện giữ 512 mẫu gần nhất.

1.2. VisionTracker

VisionTracker sử dụng cv2.VideoCapture(0) để lấy webcam mặc định. MediaPipe chạy trên luồng chính; DeepFace chạy nền, lấy bản sao frame mỗi 0,5 giây. Giải pháp đa luồng giúp camera không bị chặn bởi thời gian suy luận cảm xúc.

1.3. main_fusion.py

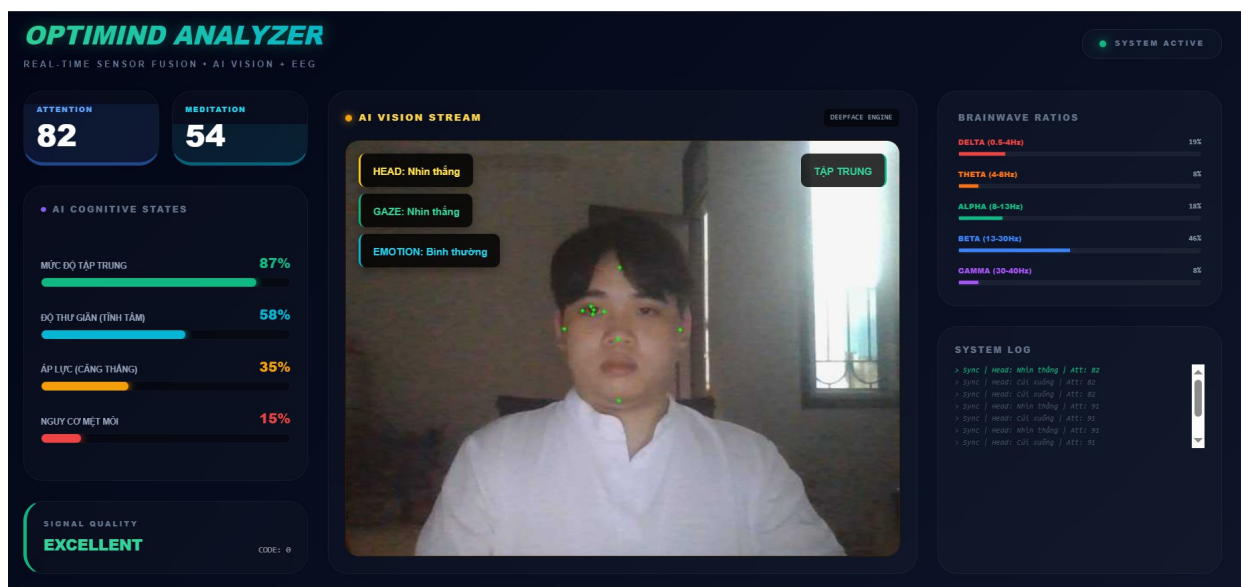
main_fusion.py nạp model và encoder, lấy snapshot EEG và camera, tạo vector 10 phân tử, gọi model.predict() và phát payload. Dữ liệu số học được phát mục tiêu 25 gói/giây; ảnh được lấy mẫu xuống 10 FPS bằng biến last_frame_send_time.

2. Triển khai Backend

Backend Node.js khởi tạo HTTP Server và Socket.IO. Khi nhận sensor_data từ Edge, server phát lại cho các trình duyệt đang kết nối và lưu một phần dữ liệu vào MongoDB. Các REST API phục vụ ghi label, lấy lịch sử, xuất báo cáo và xuất dataset huấn luyện.

3. Triển khai Frontend

Frontend gồm Master Dashboard, Calibration Lab, Dataset Lab và chức năng xuất dữ liệu. Dashboard hiển thị Attention, Meditation, signal, waveform, camera, Head Pose, Gaze, Emotion và final_state. Phần biểu đồ năng lượng sử dụng trực tiếp tám trường ASIC_EEG_POWER.



Thiết kế giao diện Dashboard

4. Huấn luyện mô hình

Tập dữ liệu sử dụng có 4.084 bản ghi và 12 cột, trong đó mô hình dùng 10 đặc trưng đầu vào và một cột nhãn mục tiêu. Sau khi bổ sung các trạng thái DeepFace, encoder Emotion có thể nhận đủ bảy biểu cảm của camera.

```

1. Đang tải dữ liệu từ file Excel CSV...
Tìm thấy tổng cộng 4084 dòng dữ liệu.
2. Đang chuẩn hóa dữ liệu (Encoding)...
3. Đang huấn luyện Mô hình AI (Random Forest)...

✅ ĐÃ HUẤN LUYỆN XONG!
🎯 Độ chính xác của AI đạt: 94.98%

📄 Báo cáo chi tiết :
           precision    recall  f1-score   support

Sao nhãng      0.94      0.96      0.95        405
Tập trung      0.96      0.94      0.95        412

 accuracy
macro avg      0.95      0.95      0.95        817
weighted avg    0.95      0.95      0.95        817

📁 Đã xuất mô hình thành công thành file: 'optimind_ai_model.pkl'

```

Kết quả training_model

Thông số	Giá trị
Tổng số bản ghi	4.084
Số đặc trưng đầu vào	10
Số cây Random Forest	100
Tỷ lệ train/test	80% / 20%

Cấu hình huấn luyện

5. Kết quả đánh giá

5.1. Kết quả trên phép chia ngẫu nhiên

Với `random_state = 42` và phép chia ngẫu nhiên theo dòng, mô hình đạt accuracy 100% trên 817 mẫu kiểm thử. Ma trận nhầm lẫn không có mẫu sai trong phép thử này.

Kết quả 95% không đồng nghĩa mô hình đã đạt độ chính xác tương đương trong môi trường thực tế. Các bản ghi liên tiếp trong cùng phiên có thể rất giống nhau và xuất hiện đồng thời ở train và test. Để đánh giá khả năng tổng quát, cần bổ sung `participant_id`, `session_id` và chia dữ liệu theo phiên hoặc theo người dùng.

5.2. Đánh giá luồng thời gian thực

. Tần suất xử lý và truyền dữ liệu

Thành phần	Nhịp xử lý mục tiêu	Nhận xét
Raw EEG / sensor_data	25 gói/giây	Phù hợp để waveform cập nhật mượt; các chỉ số eSense có thể lặp giữa các gói.
Camera Base64	10 ảnh/giây	Giảm băng thông so với gửi ảnh trong mọi gói.
DeepFace	Khoảng 2 lần/giây	Chạy nền, phù hợp vì emotion không cần cập nhật theo FPS camera.
Lưu MongoDB	Nên thấp hơn luồng hiển thị	Tránh ghi 25 document/giây nếu không cần phân tích raw dài hạn.

V. KẾT LUẬN

1. Kết quả đạt được

- Xây dựng được luồng AIoT khép kín từ TGAM và webcam đến Edge, Backend, MongoDB và React Dashboard.
- Đọc được Attention, Meditation, chất lượng tín hiệu, raw EEG và tám dải ASIC_EEG_POWER.
- Phân tích được hướng đầu, hướng mắt và bảy biểu cảm DeepFace bằng cơ chế đa luồng.
- Huấn luyện và nạp được Random Forest phân loại hai trạng thái; hỗ trợ gán nhãn và xuất dataset.
- Thiết kế được giao diện hiển thị EEG, camera và trạng thái nhận thức theo thời gian thực.

2. Hạn chế

- Thiết bị EEG một kênh vùng trán nhạy với chuyển động, nhấn trán và chất lượng tiếp xúc điện cực.
- TGAMReader chưa tự reconnect sau lỗi Serial và chưa đóng COM bằng hàm stop() rõ ràng.
- Độ chính xác 95% mới chỉ phản ánh phép chia ngẫu nhiên của tập dữ liệu hiện tại; chưa đủ bằng chứng cho khả năng tổng quát.
- Các công thức Focus, Stress và Fatigue là heuristic trực quan, không phải chỉ số chẩn đoán y khoa.

3. Hướng phát triển

- Bổ sung EAR, PERCLOS, MAR, head pitch và blink rate để phát hiện buồn ngủ, nhắm mắt, ngáp và gật đầu.
- Thu dữ liệu từ nhiều người và nhiều phiên; chia train/test theo participant_id hoặc session_id.
- Thay LabelEncoder bằng pipeline tiền xử lý có OneHotEncoder (handle_unknown = "ignore").
- Bổ sung timestamp nguồn, sequence, session_id, xác thực Edge và truyền WSS/HTTPS.
- Tăng độ bền TGAMReader bằng lock, stop(), reconnect và theo dõi thời điểm gói cuối.
- Giảm lưu ảnh hoặc chỉ lưu metadata nhằm bảo vệ quyền riêng tư và giảm dung lượng.

TÀI LIỆU THAM KHẢO

[1] NeuroSky, “ThinkGear Communications Protocol”, tài liệu giao tiếp và định dạng gói TGAM.

https://developer.neurosky.com/docs/doku.php?id=thinkgear_communications_protocol

[2] Google, “MediaPipe Face Mesh”, tài liệu nhận diện điểm mốc khuôn mặt.

<https://developers.google.com/mediapipe>

[3] S. I. Serengil và A. Ozpinar, “DeepFace: A Lightweight Face Recognition and Facial Attribute Analysis Framework”, thư viện DeepFace. <https://github.com/serengil/deepface>

[4] Scikit-learn, “RandomForestClassifier”, tài liệu mô hình Random Forest. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

[5] Socket.IO, “Socket.IO Documentation”, tài liệu truyền dữ liệu real-time.

<https://socket.io/docs/v4/>

[6] MongoDB, “MongoDB Documentation”, tài liệu cơ sở dữ liệu MongoDB.

<https://www.mongodb.com/docs/>

[7] OpenCV, “OpenCV Documentation”, tài liệu xử lý ảnh và camera.

<https://docs.opencv.org/>